Graph systems as precedents for all-brush painting

Research note, not an architecture decision. Painter presently stores immutable sampled strokes in document coordinates; deterministic pencil splats are derived from those samples, and tiled raster images are cached for display. The brief's future ambition is that **every** brush be graph-defined; the brush graph is not implemented. A separate future composition graph is also mentioned. Those are different jobs: one creates marks during a stroke; the other combines document/canvas outputs [1, 2]. Below, "documented" means behavior stated by a product's own manual or creator; "proposed" means a possible Painter design, not an existing feature.

Four graph boundaries worth distinguishing

Blender (documented). Its shared node-editor interaction vocabulary spans four **different** trees: geometry for procedural modeling, shader for materials, compositor for rendered images, and texture for custom textures [3]. A shader socket conveys a lighting-interaction description, not merely a surface color; compositor nodes process one image/frame at a time [4, 5]. The geometry field is a reusable instruction evaluated for each element in the context of a downstream geometry operation, not a precomputed array; the same upstream field can give different values at two downstream sites [6]. Socket color identifies data type; socket shape additionally distinguishes single values, per-element fields, grids, lists, and dynamic structures. A scalar connected to a field input broadcasts over elements [7]. Thus a common **editor** need not mean a common evaluation domain. Blender groups package reusable subgraphs with exposed sockets; recursive groups are forbidden [8].

Our Paint (documented, at the available level of evidence). Its creator describes an infinite-canvas painting application with a node-based brush engine offering flexible but constrained control of brush dynamics, GPU canvas operations, and a canvas that records pigment information per pixel [9]. A v0.5 announcement names a dual-component pigment canvas, brush depletion, and oil/gouache/G-pen brushes [10]. These sources establish a **real brush-node product**, unlike extrapolating from a generic image-compositing graph. They do **not** specify individual node classes, socket types, execution cadence, persistence, compiler, or whether oil/gouache each consist entirely of nodes. No such internals should be attributed to it on this evidence. Its pigment-per-pixel canvas is not evidence that Painter's immutable stroke representation already supports the same interactions.

TouchDesigner (documented). Seven operator families partition work: TOP images, CHOP sampled channels (including audio/control), POP GPU point/geometry data, DAT text/tables, MAT materials, SOP legacy geometry, and COMP containers. Ordinary wires connect only within a family; cross-family data can pass through a reference/export or a conversion operator, not an arbitrary same-looking cable [11, 12]. "Cook" is demand-driven: a node needs both a request from a dependent/viewer/output and a reason such as changed input/parameter or time dependence; a changed upstream constant is dirty but need not cook immediately. Visible node viewers themselves can request cooking [13]. Feedback TOP explicitly sources a target TOP downstream in the feedback network and has reset/bypass controls [14].

Modular audio reference (identity uncertain). Searches for "VX Workstation" did not establish a particular modular DAW; do not identify it as a product. **Bespoke Synth** is a defensible nearby reference because its own site calls it a modular DAW, describing patchable separate modules and an exploration/jamming emphasis rather than a global timeline [15]. Its manual documents patching a module by dragging a cable into empty space to spawn a connecting module and shift-inserting on a cable [16]. **VCV Rack** instead documents a virtual modular audio rack (including audio-device sample rate, block size, and latency), not proof of the requested DAW identity [17]. **Bitwig's Grid** is

an embedded modular sound-design environment with distinct Poly/FX/Note Grid devices; it explicitly permits any signal output to connect to any signal input and describes signals as stereo [18]. This is a deliberate alternative to Blender/TouchDesigner's visibly typed or family-separated edges, not evidence that an untyped Painter graph would be safe.

Execution phases, contracts, and the stroke boundary

Proposed for investigation, not a decision: separately describe input acquisition, resampling/smoothing, per-stroke state initialization, per-sample/per-dab attribute generation, mark emission, document persistence, and later tile rasterization. Their possible outputs have different meanings: stylus events versus stable document-space samples; a scalar pressure versus a scalar field over dabs; pigment or image data versus an immutable mark record. Such phases could make the existing pencil generator one graph-defined brush without making the composition graph a brush graph [1, 2, 6, 7]. Determine empirically whether a graph should yield a bounded mark stream, a raster patch, or both; treating all of them as interchangeable values would conceal replay, hit-testing, and tile-cache invalidation costs.

Proposed socket contract: make domain/cadence and units part of the meaning of an edge (e.g., one stroke, one sample, one dab, one tile; document length versus screen pixels; color in a stated color space), in addition to type and multiplicity. A field-like expression can defer per-dab evaluation until its consumer is known; an explicit "capture" can stabilize values when the consumer's context changes. This follows the **reason** for Blender's fields and captured attributes [6], not a claim that Painter needs Blender's exact geometry-domain API. A visible adapter for pressure-to-radius, color-space changes, or mark-to-image rasterization would make lossy or expensive boundaries inspectable. Blender permits implicit float/color links, float-to-integer truncation and unit-sensitive angle links, while its explicit Implicit Conversion node can expose otherwise hidden conversions [7, 19]. TouchDesigner takes a stricter route between families: for example CHOP to TOP maps channel samples into image pixels under an explicit data-format/layout choice [11, 12].

State and feedback: a repeat zone performs multiple iterations in one evaluation, carrying values between iterations; a simulation zone carries state between animation frames, has a clock/cache, and is unavailable in Blender's Geometry Nodes Tool context [20, 21]. TouchDesigner's Feedback TOP makes delayed image feedback an explicit stateful operator [14]. For painting, "previous dab," "previous stroke," and "previous raster tile" are different clocks; feedback without a declared clock, initial value, reset, and replay policy threatens determinism of stored strokes. **Proposed:** test a narrowly scoped stroke-local delay/state boundary before allowing feedback from the rendered canvas; once canvas reads affect marks, order of edits and cache state become observable. A physical pigment simulation needs an evolving pigment reservoir/canvas state and conservation/transfer assumptions; a smear, speck, or grain shader that only **looks** wet is a visual approximation, not simulation fidelity [9, 10].

Authoring and performance implications

Proposed UX question: favor a paint-ready default over making artists wire a graph before drawing; make the graph editable through presets, discoverable inputs, annotated units, live stroke preview, and inspectable intermediate outputs. Blender documents node previews and nested groups [7, 8]; Bespoke documents cable-to-search insertion [16]; Bitwig describes per-module interactive help and an inspector showing incoming/outgoing signals [18]. These are distinct precedents, not evidence of Painter's present UI. When previewing, remember that TouchDesigner can pull work simply because a viewer is visible [13]: a per-node thumbnail at every dab may multiply evaluations rather than merely display already-computed values.

Proposed performance experiment: profile a naive per-dab interpreter against a preplanned/compiled pure subgraph with identical sampled strokes and seed; separately measure stroke generation, persisted representation size, viewport cache misses, and tile redraw. Evaluate domain-specific caching: one constant per stroke need not be recomputed per splat; fields that depend on changing geometry/context cannot be cached just because their **graph wiring** is unchanged [6]. Dirty and pull-driven evaluation can avoid offscreen preview work [13], but should not make a saved stroke depend on whether its graph editor was open. Stateful wet-media processing or raster feedback would require a separate cost/replay model; this research does not establish that GPU evaluation, shader compilation, or arbitrary cross-domain conversion is necessary for Painter.

Interesting finds and cautions

- **Blender, documented:** Blender has Geometry, Shader, Compositor and legacy Texture node editors, not one universal canvas graph [3][4]. The current manual calls texture nodes legacy, says node-based textures now chiefly serve brushes, and instructs users to choose Brush in the Texture Type selector, create a texture via the data-block menu, then enable Use Nodes before adding nodes [22]. Geometry Nodes also has simulation zones with feedback across animation frames, but the manual says they cannot be used in the Tool context and are tied to the animation frame/cache workflow—not an in-stroke brush feedback guarantee [20]. Nodes have socket types and automatic compatible conversions [7][19]. This is evidence for a typed editor and context-specific execution, not for claiming Blender can already run a Painter stroke graph.
- **Interesting find — Blender simulation is context-restricted.** Simulation zones explicitly cannot run in Geometry Nodes Tool context, despite working in modifier context [20]. A reusable node-editor widget does not guarantee every stateful node is valid in every interactive context.
- **Interesting find — TouchDesigner viewers can change the cost of a network.** Viewing an intermediate node can create a cook request independently of the final output [13]. An always-live brush-node inspector could change latency without changing a saved brush definition.
- **Interesting find — Bitwig chooses deliberately permissive wiring.** Its claim that any Grid signal connects anywhere and that every signal is stereo [18] is useful counterevidence to a universal "typed sockets are necessary" rule; painting still needs an explicit story for unlike lifetimes, spatial domains, and replay.

Sources

[1] Painter project brief, "Node Graph Brushes" and "Node Graph Composition": `docs/project-brief.md` (local project source).

[2] Painter architecture, "Canvas Architecture": `docs/architecture.md` (local project source).

[3] Blender Manual, "Nodes": docs.blender.org/manual/en/latest/interface/controls/nodes/index.html.

[4] Blender Manual, "Shader Nodes: Introduction": docs.blender.org/manual/en/latest/render/shader_nodes/introduction.html.

[5] Blender Manual, "Compositing: Introduction": docs.blender.org/manual/en/latest/compositing/introduction.html.

[6] Blender Manual, "Fields": docs.blender.org/manual/en/latest/modeling/geometry_nodes/fields.html.

[7] Blender Manual, "Node Parts" (sockets, shapes, conversions, previews): docs.blender.org/manual/en/latest/interface/controls/nodes/parts.html.

[8] Blender Manual, "Node Groups": docs.blender.org/manual/en/latest/interface/controls/nodes/groups.html.

[9] Wu Yiming, Our Paint creator's project page: ourpaint.handmade.network.

[10] Wu Yiming, "Our Paint v0.5 is out NOW!", release notes: chengdulittlea.itch.io/ourpaint/devlog/993181/our-paint-v05-is-out-now.

[11] Derivative, "Operator Family": docs.derivative.ca/Operator_Family.

[12] Derivative, "CHOP to TOP": docs.derivative.ca/CHOP_to_TOP.

[13] Derivative, "Cook": docs.derivative.ca/Cook.

[14] Derivative, "Feedback TOP": docs.derivative.ca/Feedback_TOP.

[15] Bespoke Synth, creator's product overview: bespokesynth.com.

[16] Bespoke Synth, creator's reference manual, "Basics": bespokesynth.com/docs/index.html.

[17] VCV Rack, "Core Modules", Audio interface: vcvrack.com/manual/Core.

[18] Bitwig, "The Grid": bitwig.com/the-grid.

[19] Blender Manual, "Implicit Conversion": docs.blender.org/manual/en/latest/modeling/geometry_nodes/utilities/implicit_conversion.html.

[20] Blender Manual, "Simulation Zone": docs.blender.org/manual/en/latest/modeling/geometry_nodes/simulation/simulation_zone.html.

[21] Blender Manual, "Repeat Zone": docs.blender.org/manual/en/latest/modeling/geometry_nodes/utilities/repeat_zone.html.

[22] Blender Manual, "Texture Node Editor: Introduction": docs.blender.org/manual/en/latest/editors/texture_node/introduction.html.